SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Ashwanth SP/192126002 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



CO2 – SCENARIO-BASED ANSWERS

1. AI-Powered Drones for Emergency Medicine Delivery

i. Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search chooses the next location using a heuristic estimate of how close that location appears to the destination. In this scenario, the drone can use straight-line distance as its heuristic. At each step, it prefers the available point that seems nearest to the hospital or delivery location. This can help it find a route quickly, which is valuable when emergency medicines must reach people without delay.

Its main strength is speed: it often explores fewer locations than an uninformed search because it uses the destination estimate to guide the search. However, it considers the heuristic rather than the full travel cost already incurred. A route that looks short on the map may pass through strong winds, flooded areas or difficult terrain. Therefore, Greedy Best-First Search does not guarantee the lowest-cost or safest route. Its decision can also become outdated when a path is reported blocked.

ii. A* and the balance between cost and heuristic

A* evaluates each candidate route using $f(n) = g(n) + h(n)$. Here, $g(n)$ is the actual cost from the starting point to the current location, including travel time and other defined penalties, while $h(n)$ estimates the remaining cost to the destination. The drone can assign higher costs to risky terrain, poor weather or restricted flight areas. A* therefore considers both the progress already made and the estimated distance still to travel.

When the heuristic is admissible, meaning it does not overestimate the remaining minimum cost, and the search uses suitable cost assumptions, A* can find an optimal route in a stable search graph. In practice, the drone must also account for safety constraints; the mathematically shortest route should not be selected if it violates a safe operating limit.

iii. Effect of heuristic accuracy

A useful heuristic directs the search toward promising routes. Straight-line distance is easy to calculate, but it may be a poor estimate when the drone must detour around buildings, restricted zones or severe weather. In Greedy Best-First Search, an inaccurate heuristic can lead the drone toward a route that looks close but is expensive or unsafe. In A*, a more accurate admissible heuristic can reduce unnecessary exploration while preserving optimality. An overestimating heuristic may make A* faster in some cases, but it can remove the guarantee of finding the optimal route.

iv. Dynamic changes and blocked paths

A blocked path changes the set of routes available to the drone. If the search is based on an old map, it may waste time exploring an unusable path or attempt to follow a route that is no longer safe. The drone should receive updates from sensors, weather services and emergency-control systems, invalidate affected edges, and re-plan from its current position. Incremental or repeated

planning can be more practical than restarting a large search from the beginning after every small change.

v. Recommended algorithm

A* with frequent map updates and explicit risk costs is the most suitable starting choice for this scenario because it balances known travel cost with an estimate of the remaining journey. For urgent delivery, the system should use a time limit for each planning cycle and maintain a safe fallback route. If conditions change, it must re-plan immediately rather than follow the original route blindly. Safety restrictions should be treated as hard constraints, while delay and energy use can be represented as costs. Greedy Best-First Search may be useful for a quick initial estimate, but A* is preferable when route quality and safety matter alongside speed.

2. Smart-City Traffic Signal Optimization

Formulation as an optimization problem

The system can represent each traffic signal plan as a candidate solution. Variables may include green-light duration, phase order, offsets between nearby intersections and permitted turning movements. The objective is to minimize a combined cost such as total vehicle waiting time, fuel consumption and congestion. These factors can be given weights based on city priorities. Constraints include minimum and maximum green times, pedestrian crossing time, safe phase separation and emergency-vehicle requirements. Traffic sensors provide updated information, so the solution must be revised as demand changes.

i. Hill Climbing and its limitations

Hill Climbing begins with a signal plan and repeatedly examines neighbouring plans. A neighbour might increase a green phase by a few seconds or adjust the offset between two intersections. If the change improves the objective function, the system adopts it and continues. This is simple to implement and can improve a plan quickly. Its limitation is that it generally makes local decisions and may stop when none of the immediately available changes looks better, even though a much better overall plan exists elsewhere in the search space.

ii. Local maxima, plateaus and ridges

A local optimum occurs when a plan is better than its nearby alternatives but is not the best plan overall. For example, improving one intersection may increase queues at the next intersection, making the system settle on a locally good arrangement. A plateau is a region where several plans have almost the same score, so the algorithm receives little guidance about which direction to move. A ridge occurs when improvement requires a coordinated change across several variables; changing one signal at a time may appear unhelpful even when changing several signals together would reduce congestion.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse neighbouring solution early in the search. This controlled willingness to move away from a local optimum can help it explore other regions. As the temperature parameter decreases, the probability of accepting worse solutions also decreases,

allowing the search to become more selective. A Genetic Algorithm instead maintains a population of candidate plans. Selection favours better plans, crossover combines parts of two plans, and mutation introduces new variations. These mechanisms provide broader exploration than basic Hill Climbing, although both methods need carefully designed parameters and evaluation functions.

iv. Recommended approach

For a city with hundreds of intersections, a Genetic Algorithm can be suitable for offline or periodic optimization because it explores many candidate plans and can represent coordinated changes across intersections. A practical system could generate candidate plans using a Genetic Algorithm, evaluate them using traffic simulation, and then apply only safe, validated changes. For real-time operation, local adjustments can be made between larger optimization runs. The objective should include waiting time, fuel use and queue spillback, while safety and pedestrian requirements remain mandatory constraints. Continuous sensor feedback is essential because even a strong plan can become ineffective after an accident or sudden traffic surge.

3. Autonomous Mars Rover in Unknown Terrain

i. Why online search is required

Offline search assumes that the agent has a sufficiently complete model of the environment before planning. A Mars rover cannot rely on this assumption because its cameras and sensors reveal only part of the terrain, and distant areas may be hidden by rocks, slopes or dust. An online search agent alternates between taking an action, observing the result and updating its plan. This is appropriate for exploration because the rover learns about the environment as it moves.

ii. Partial observability and dynamic conditions

The rover may not know the exact depth of a crater, the stability of a surface or whether a path is safe until it gathers more sensor data. Sensor readings can also be incomplete or noisy. Although the Martian landscape may change more slowly than a busy road, dust, lighting, equipment condition and communication delays can affect decisions. A wrong movement can damage the rover or make a sample location unreachable, so the system must represent uncertainty and avoid routes whose risk is beyond an acceptable threshold.

iii. Building and updating an internal model

The rover can maintain a local map containing known free areas, obstacles, uncertain regions and possible routes. After each movement, it combines camera images, range measurements, wheel odometry and other available sensor readings to update its estimate of position and nearby terrain. New observations can change a cell from unknown to traversable or mark it as hazardous. The rover then plans again using the updated map. It should also record the confidence of observations, since a low-confidence reading should not be treated as certain evidence that a route is safe.

iv. Online, uninformed and informed search

Uninformed methods such as breadth-first search explore without using a goal-distance estimate, so they may spend time examining many irrelevant locations. Informed methods use a heuristic, such as estimated distance to a target, to guide exploration. However, a heuristic alone does not solve the problem of an incomplete map. Online search is distinguished by the way the agent acts and learns during the search. An online rover can combine heuristic guidance with cautious exploration and re-planning as new information arrives.

v. Recommended approach

A suitable solution is an online, heuristic-guided planner that maintains a local map, assigns costs to slope and hazards, and replans after every meaningful observation. The rover should use conservative movement limits and stop to gather more information when uncertainty is high. Its priorities should be safety first, then mission progress and energy efficiency. This approach is more adaptable than a fixed offline route because it can respond to discoveries without assuming that the entire Martian environment is known in advance.

4. University Examination Timetabling as a CSP

i. Variables, domains and constraints

A Constraint Satisfaction Problem (CSP) consists of variables, domains of possible values and constraints that specify which combinations are allowed. For this timetable, each examination can be represented by a variable. Its domain contains possible time slots and examination halls. If hall allocation is handled separately, the variable may represent only the time slot, with a second variable for the hall. The domain must exclude unavailable periods and halls that cannot accommodate the expected number of students.

Hard constraints include preventing a student from having two examinations at the same time, ensuring that a hall is not assigned to more than one examination in the same slot, respecting faculty availability and scheduling prerequisite subjects in the required order. Special accommodations may require accessible halls, extra time or particular time slots. Preferences such as avoiding too many examinations on consecutive days can be represented as soft constraints and optimized after the hard constraints are satisfied.

ii. Backtracking search

Backtracking assigns a value to one variable at a time. The system selects an examination, chooses a time and hall, and checks whether the assignment violates any constraint. If the assignment is consistent, it continues with another examination. If no available value works, it returns to an earlier variable and tries a different choice. This guarantees that an exhaustive backtracking procedure can find a solution if one exists, but the number of combinations may become very large as the number of courses and students grows.

iii. Constraint propagation and forward checking

Forward checking reduces the domains of unassigned variables whenever a new assignment is made. For example, if an examination is assigned to a particular time slot, that slot can be

removed from the domains of other examinations taken by the same students. If any unassigned examination is left with an empty domain, the system detects the conflict immediately and backtracks rather than continuing down an impossible path. Constraint propagation can therefore prevent many avoidable checks. A minimum-remaining-values rule can also select the examination with the fewest legal options first, while a degree heuristic can prioritize examinations that constrain the largest number of others.

iv. Real-world challenges and scalable strategy

Real timetabling involves incomplete enrolment data, last-minute faculty changes, room capacities, accessibility needs and student preferences. A timetable that satisfies every hard constraint may still be inconvenient, so the system must distinguish mandatory rules from preferences. For scalability, the university should combine backtracking with forward checking, intelligent variable ordering and a constraint solver. It can first generate a feasible timetable, then improve soft constraints through local search. Before publication, the timetable should be validated against the latest enrolment and room data, with a controlled process for handling changes.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. Minimax model

Minimax models a competitive game in which one player tries to maximize the outcome and the opponent tries to minimize it. The game is represented as a tree: nodes are game states, and edges are legal moves. At a maximizing node, the algorithm selects the child with the highest value; at a minimizing node, it selects the child with the lowest value. By reasoning about the opponent's likely best response, the AI avoids moves that look attractive immediately but lead to a poor position after the opponent acts.

ii. Challenges of large game trees

If the average number of legal moves is b and the search depth is d, a basic minimax search may examine on the order of b^d nodes. This grows rapidly even for moderately complex games. A real-time strategy game may also contain many units, resources, possible actions and changing objectives. Searching the entire tree is often impossible within a strict response deadline, so the AI needs pruning, depth limits, good move ordering and an evaluation function that estimates the value of states that are not terminal.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning produces the same minimax decision as a full search when applied correctly, but it avoids exploring branches that cannot affect the final choice. Alpha is the best value the maximizing player can guarantee so far, while beta is the best value the minimizing player can guarantee. When a branch proves that alpha is at least beta, further exploration of that branch is unnecessary. For example, if the maximizing player already has an option worth 8, a minimizing branch that is guaranteed to produce a value no greater than 5 cannot become the preferred choice. Good move ordering allows pruning to occur earlier, making deeper search possible with the same time budget.

iv. Evaluation function

A practical evaluation function could combine resource advantage, number and strength of units, territory controlled, defensive position, objective progress and estimated damage potential. Each factor receives a weight based on its importance to winning. For example, resource collection matters early in a match, while protecting a critical base may become more important when the opponent is close to winning. The function should avoid double-counting related factors and should be tested against different strategies. A poor evaluation function can cause the AI to choose a state that scores well numerically but is strategically weak.

v. Balancing optimality and real-time performance

The recommended strategy is depth-limited Minimax with Alpha-Beta pruning, effective move ordering and a carefully tested evaluation function. Iterative deepening can search depth one, then depth two and so on, retaining the best move found if the time limit expires. The AI should also allocate more computation to important decisions and use a safe fallback action when time is nearly exhausted. This method does not guarantee a perfect move when the search is cut off, but it provides a practical balance between decision quality and responsiveness.

Conclusion

These scenarios show that an AI search technique should be selected according to the structure of the problem, the quality of available information and the time available for a decision. A* is useful when route cost and a heuristic must be balanced; broader optimization methods can help traffic systems escape local optima; online search supports exploration in unknown environments; CSP methods manage timetable constraints; and Minimax with Alpha-Beta pruning supports competitive game decisions. In every case, the algorithm works best when its cost function, constraints and model are designed around the real requirements of the application.